

GUÍA DE APRENDIZAJE

Control de Versiones con Git

1. Introducción

Esta guía documenta el desarrollo completo de las actividades propuestas para el aprendizaje de Git: la reflexión inicial, la apropiación práctica de los comandos git clone, git status, git add . y git log, el manejo de ramas (branches), y la preparación de la actividad de transferencia (sustentación).

2. Actividades de Reflexión

A continuación se presentan las respuestas individuales a las preguntas orientadoras, como base para la socialización grupal.

1. Tres compañeros compartiendo código por correo o USB: ¿qué problemas podrían presentarse?

Primero, es muy fácil sobrescribir por accidente el trabajo de otro compañero si dos personas modifican el mismo archivo al mismo tiempo, porque no hay forma de combinar automáticamente los cambios. Segundo, se pierde la trazabilidad: nadie sabe con certeza cuál es la versión más reciente ni quién hizo cada cambio, y suelen aparecer archivos como "proyecto_final_v2_definitivo.zip" que generan confusión. Tercero, no existe una forma confiable de recuperar una versión anterior si algo se daña, ya que depende de que alguien haya guardado copias manuales, lo cual rara vez ocurre de forma ordenada.

2. Un cambio de hace cinco días dañó una funcionalidad. ¿Cómo se resolvería sin control de versiones?

Sin una herramienta como Git, tendría que revisar manualmente el código completo buscando el error, comparando "a ojo" contra copias de seguridad si existieran, lo cual es lento y propenso a errores. Probablemente terminaría reescribiendo partes del código desde cero o perdiendo cambios válidos en el proceso. Con Git, en cambio, bastaría con usar git log para ubicar el commit donde se introdujo el problema y revisar exactamente qué cambió, lo que ahorra horas de trabajo.

3. ¿Por qué las empresas de tecnología exigen conocimientos de Git para casi cualquier vacante de desarrollo?

Porque Git es independiente del lenguaje de programación: se usa igual en proyectos de Java, Python, JavaScript o cualquier otro, y es la base de la colaboración en equipo. Sin Git no es posible trabajar de forma ordenada con otras personas, integrar código en producción con confianza, ni mantener un historial que permita auditar y corregir errores. Es una habilidad transversal tan básica como saber usar la terminal.

4. Situación personal donde se perdió trabajo o se sobrescribió un archivo importante.

En un trabajo académico grupal, dos integrantes editamos el mismo documento de código al mismo tiempo y, al guardar por USB, la última copia sobrescribió por completo los cambios del otro compañero sin que nos diéramos cuenta hasta la entrega. Tuvimos que rehacer esa parte del trabajo bajo presión de

tiempo. Esa experiencia es justamente el tipo de problema que un sistema como Git evita, gracias al historial de commits y a la posibilidad de fusionar cambios de forma controlada.

4. Actividades de Apropiación del Conocimiento

Registro paso a paso de la ejecución de cada comando sobre un repositorio de práctica clonado desde GitHub.

4.1 git clone

Se ubicó el repositorio en GitHub, se copió la URL HTTPS desde el botón "Code" y se ejecutó el clonado en la terminal:

```
$ git clone https://github.com/usuario/nombre-repositorio.git
$ cd nombre-repositorio
$ ls
```

Git trajo el historial completo de commits y creó automáticamente la conexión remota "origin" hacia el repositorio original, necesaria para usar push y pull más adelante.

4.2 git status

Se creó un archivo de prueba (prueba.txt) y se consultó el estado del repositorio:

```
$ git status
Untracked files:
  prueba.txt
```

Como es un archivo nuevo, apareció en rojo bajo "Untracked files". Este comando se repitió después de cada paso importante (crear archivo, hacer add, hacer commit) para verificar el estado antes de continuar, siguiendo la buena práctica de consultarlo con frecuencia.

4.3 git add .

```
$ git add .
$ git status
Changes to be committed:
  new file:   prueba.txt
```

Después de ejecutar git add ., el archivo pasó al área de preparación (staging area) y apareció en verde bajo "Changes to be committed", listo para el próximo commit.

4.4 git log

Se confirmó primero el cambio preparado con un commit, y luego se consultó el historial:

```
$ git commit -m "Agrega archivo de prueba"
$ git log
$ git log --oneline
```

git log mostró el hash único del commit, el autor, la fecha y el mensaje descrito. La versión --oneline resume cada commit en una sola línea, forma muy usada en la práctica profesional para revisar el historial rápidamente.

4.5 Creación y uso de ramas (branches)

```
$ git branch
$ git branch feature-login
$ git checkout feature-login
# o en un solo paso:
$ git checkout -b feature-login

# dentro de la rama:
$ git status
$ git add .
$ git commit -m "Agrega estructura inicial del formulario de login"
$ git log --oneline

# regreso y fusión:
$ git checkout main
$ git merge feature-login
```

Se verificó la rama activa con `git branch` (marcada con asterisco), se creó la rama `feature-login`, se trabajó dentro de ella aplicando el flujo `status` → `add` → `commit`, y finalmente se regresó a `main` para fusionar los cambios con `git merge`. Este flujo permite desarrollar una funcionalidad de forma aislada sin poner en riesgo la estabilidad del código principal.

5. Actividad de Transferencia (Sustentación)

Para la sustentación en vivo ante el instructor se seguirá, en orden, el procedimiento completo: clonar el repositorio, verificar su estado, crear un archivo relacionado con una funcionalidad ficticia, prepararlo con `git add .`, confirmarlo con un mensaje de `commit` claro, consultar el historial, crear y cambiar a una nueva rama, realizar al menos un cambio y `commit` dentro de ella, y finalmente regresar a `main` y fusionar los cambios. Cada comando se explicará en voz alta indicando su propósito dentro del flujo de trabajo.

5.4 Respuestas a las preguntas orientadoras de cierre

¿Qué hubiera pasado si se hubiera olvidado ejecutar `git add .` antes del `commit`?

El `commit` se habría realizado, pero sin incluir los cambios nuevos, ya que `git commit` solo confirma lo que está en el área de preparación (staging area). Los archivos modificados seguirían apareciendo como "modified" o "untracked" en `git status`, y sería necesario ejecutar `git add .` y volver a hacer `commit` para que esos cambios quedaran realmente registrados en el historial.

¿Por qué no se debería hacer `commit` directamente sobre la rama `main` en un equipo real?

Porque `main` representa la versión estable del proyecto, la que normalmente está en producción o lista para entregarse. Hacer `commits` directos allí expone ese código a errores no probados y puede afectar a todo el equipo de inmediato. Trabajar en ramas separadas permite desarrollar y probar una funcionalidad de forma aislada, y solo integrarla a `main` mediante un `merge` cuando ya está validada.

¿Cómo permite `git log` identificar quién introdujo un error específico?

Porque cada `commit` registrado en `git log` incluye el autor, la fecha y un mensaje descriptivo del cambio. Si una funcionalidad deja de funcionar, el equipo puede revisar el historial y ubicar el `commit` donde se

modificó esa parte del código, ver exactamente qué se cambió y quién lo hizo, lo que facilita corregir el error y conversar directamente con la persona responsable si es necesario.

¿Qué relación hay entre el uso de ramas y la calidad del software entregado a un cliente?

El uso de ramas evita que código incompleto o con errores llegue a la versión que el cliente recibe, porque cada funcionalidad se desarrolla y prueba de forma aislada antes de fusionarse a main. Esto reduce la probabilidad de que un cambio dañe otras partes del sistema, permite revisar el código antes de integrarlo y facilita que varios desarrolladores trabajen en paralelo sin pisarse el trabajo, lo que en conjunto se traduce en un producto final más estable y confiable.

Nota final

El objetivo de este trabajo no fue memorizar comandos, sino comprender el propósito de cada uno dentro del ciclo de vida del desarrollo de software: git clone para obtener el proyecto, git status para conocer su estado, git add . para preparar cambios, git commit y git log para registrar y consultar el historial, y las ramas para permitir el trabajo colaborativo sin arriesgar la estabilidad del código principal.